

HOMework 4 - Nbody Solver

Due: Monday Dec. 8

Notebook:

nbody-MS-2025.ipynb

OVERVIEW

Learn to use and modify the brute-force Nbody solver on the Python notebook you were provided. A Nbody solver is a way to solve gravitational interactions for a large set of particles that sample a potential complicated density field. "Brute-force" refers to the fact that in this version we calculate all the interactions between all particles pair by pair (pairwise calculation). With the current modules you can: observe the collapse of a dark matter halo (or a cluster of stars) and the evolution towards the energy balance predicted by the virial theorem. You can also include solid rotation in your initial halo.

TASKS

1) Simple Use (Difficulty: 1/4)

- > Modify the number of particles N (try 5 different N between 20 to 150): How does the computational time vary with the number of particles?
- > Modify the amount of initial solid rotation of the initial density peak How does it change its evolution? How many time steps do you need to reach virial equilibrium?
- > Modify the total mass twice and compare.
- > Modify the softening twice and compare.

2) Add expansion of the Universe (Difficulty: 2/4). Use $N=50$

- > Fill in the empty module called "scale factor" with any given model you want for the scale factor. You can use a simplified model or find a library that contains $a(t)$ for a LambdaCDM Universe.
- > Modify the equations for gravitational interactions to take into account the expansion.
- > Try plotting the results in physical coordinates, then in co-moving coordinates. What is the role of expansion?

3) Modify the Initial Conditions.

- > Modify the IC module to start with a purely radial flow of particles (you can choose the speed). What do you notice? Use=50 (Difficulty: 2/4)
- > Modify the IC module to give the halo a velocity distribution that follows a Maxwell-Boltzmann distribution at a given temperature T . Use=50. (Difficulty: 3/4)

—> Modify the IC module to include 2 nearby peaks (they can have some individual rotation if you want). Describe the merger when varying the initial distance or initial velocity between them. Use=100 (Difficulty: 3/4)

ALL PARTS BELOW ARE OPTIONAL - Difficulty: 4/4 - (Bonus homework or final project?)

4) Modify the solver to make it faster. Difficulty: 4/4 - (Bonus homework or final project?)

The present solver calculates all gravitational interactions between any pair of 2 particles. This makes it precise but painfully slow!

In modern cosmological simulations evolving > billions of particles, we use approximations to speed up the process. One example approximation consists in calculating the gravitational potential on small scales and large scales separately, only doing pairwise calculations locally, for close-by particles. Try to replicate this process:

- Decompose your volume spatially in a 3D regular, cubic grid with the same size for every pixel (cell) for simplicity (PxPxP array or matrix). Start with 1000 particles and a 10x10x10 grid.

ON LARGE SCALES:

- In each cell of the grid, store the total mass of particles in it.
- Write the Poisson equation in Fourier space (i.e for a Fourier mode) and deduce a simple relationship between the Fourier mode k of the **gravitational potential** and the corresponding Fourier mode k for the **density field**.
- Calculate numerically the Fourier series of your density field on the grid. (Using `numpy.fft` for instance)
- Deduce numerically the Fourier series of the gravitational potential field. Invert this Fourier series to obtain the real potential in each pixel. This is the smoothed, LARGE SCALE potential. Any particle in a pixel feels the potential you calculated in said pixel.

ON SMALL SCALES:

- To include a correction on small scales, Calculate the particle-particle interactions for all particles within the same pixel (or set of 27 neighboring pixels).
- Combine the two solvers wisely, making sure not to count the same interaction twice.

Now repeat at each time step (loop over time steps)!

- Does it speed up the simulation? By how much?

5) Hardest: Random Initial Density Fields of given power spectrum. (final project?)

For computational speed reasons, first modify the solver to be only 2D (only x and y dimensions) but have larger N (Difficulty: 4/4)

Modify the IC module to start from a 2D random density field with a power spectrum $P(k)$ that goes as $\sim k$ until a maximum corresponding to given wavelength of $\sim 0.5 \times$ size of the box, and decreases as k^{-3} at higher k . Ensure the continuity of $P(k)$. Choose a typical spread in density contrast of 0.1%. This is a very rough approximation of the LCDM power spectrum transferred to $z=10$.

Using Poisson equation and plugging in a generic Fourier mode k , deduce the **initial power spectrum of the potential as function of $P(k)$** . Use this new power spectrum to generate the initial potential on a regular grid $N \times N$.

Hint: You can use the the following library: <https://github.com/cphyc/EyeldGenerator> **to generate a random field on a regular grid.**

Hint 2: Think about what's the largest and smallest k we can resolve.

Assuming your initial particles are evenly distributed at the center of each pixel of the grid, Use Zeldovich approximation to assign to each particle its initial velocity (to a multiplying factor, that you may treat as a free parameter in this simple demonstration) and spatial shift after one timestep.

Good reference: <https://iopscience.iop.org/article/10.1086/497090/pdf>